



CEVA XM Training

Dec 2020

www.ceva-dsp.com

CEVA Confidential & Proprietary Information – under NDA

Copyright © 2020 CEVA, Inc.

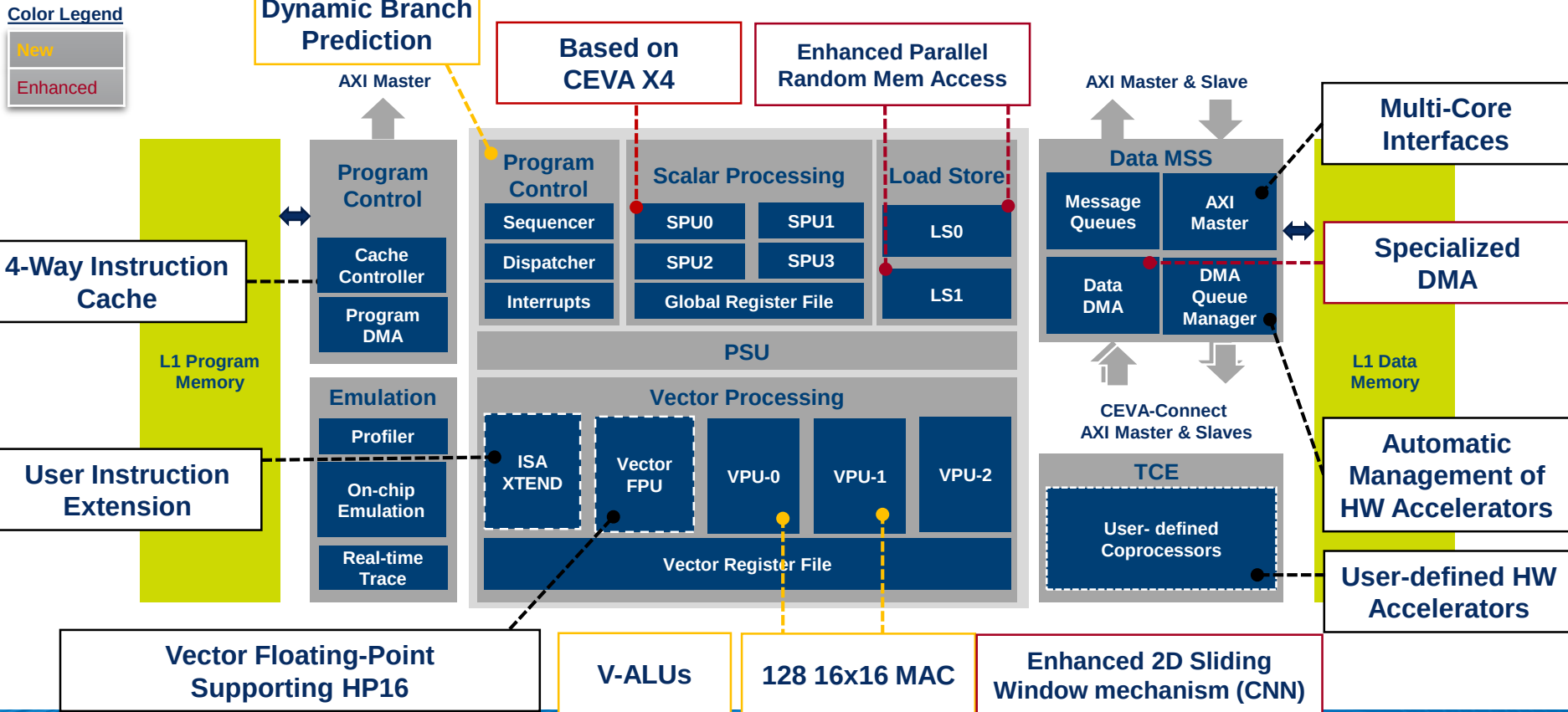


▶ Architecture features

Architecture Features

Feature	XM6
Vector size	256 bits
Vector file size	48 orthogonal
Memory BW	• 2 Load / Store units capable of • 2 x 256 bits loads • 256 bits load + 256 bit store • Additional 512 bits Store in parallel to the “regular” LD/ST units
Memory partition	16 banks of 32 bits
Scatter-Gather	32 addresses Greatly enhanced capabilities
Vector units	3
MAC units (16*16 bits)	128
ALU units (8/16/32 bits)	96 / 48 / 24

CEVA-XM6 Block Diagram



CEVA-XM6 – Load / Store

- ▶ Parallel LD/ST bank conflict is **allowed**
- ▶ VHIST Mechanism Enhancement
 - ▷ Support for 32 bits weights vs. 16bit weights in CEVA-XM4
 - ▷ Can be effectively used as an additional ST unit
 - ▶ Executes on LVPU, freeing Main VPUs to continue executing MAC operations
 - ▶ “Regular” LD/ST units can work in parallel

000 LOAD/STORE EXECUTION

LOAD	$R3 = \emptyset(R6)$
ADD	$R7 = R3 + R9$
STORE	$R4 \rightarrow \emptyset(R7)$
SUB	$R1 = R1 - R2$
LOAD	$R8 = \emptyset(R1)$

DISP MISS!

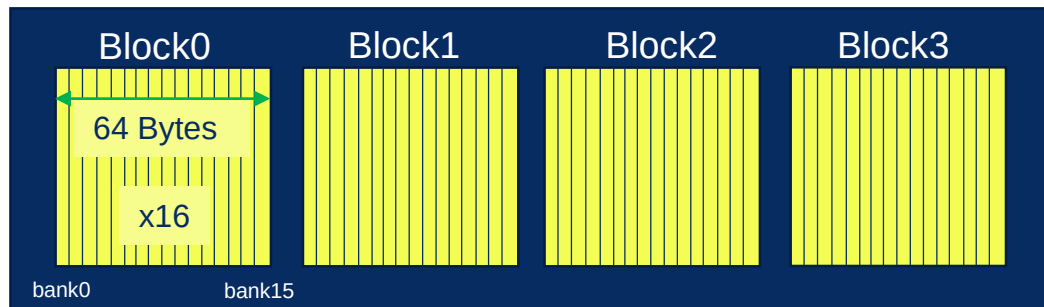
LSU and Memory sub-system

- ▶ Each LSU (Load/Store Unit) can Load / Store up to **256bits** in one cycle (=1 Vector Length)
- ▶ Total BW using the 2 LSUs is 512bits per cycle
 - ▶ **Valid** Combination:
 - ▶ LD || LD
 - ▶ LD || ST
 - ▶ **Invalid** Combination :
 - ▶ ST || ST
- ▶ Two Groups of Load / Store ISAs:
 - ▶ Regular Load/Store: VLD/VST, VLDOV (Overlap), VLDCHK (Checkered)
 - ▶ Parallel Load Store: VPLD/VPST, VBCPLD/VBCPLD (bank-coupled)

LSU and Memory sub-system

▶ CEVA-XM Memory Subsystem

- ▶ Bank width = **32 bit = 4 Bytes**
- ▶ # Banks in Block = **16**
- ▶ # Blocks in DTCM = 4
- ▶ Block Width = **512 bit = 64 Bytes**
= 2 x Vector Size

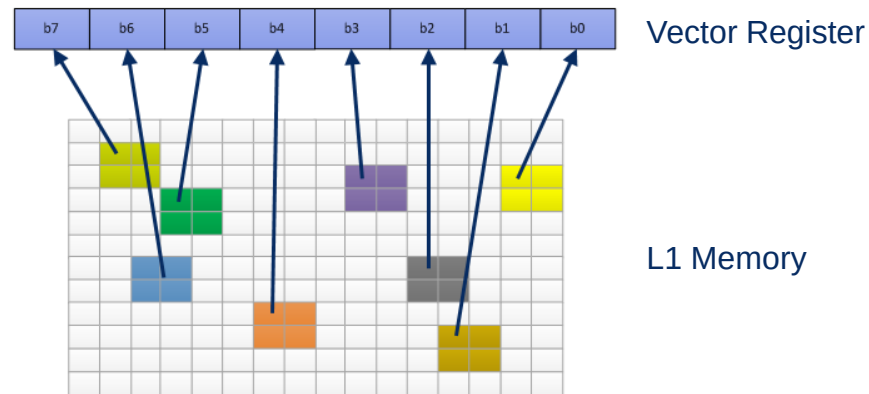


▶ LSU capabilities:

- ▶ Regular LD/ST – **Up to 2 Vectors LD/ST** in one cycle
 - ▶ from the same Block (bank 0-7, banks 8-15)
 - ▶ Or from two different blocks
- ▶ Parallel LD/ST (“Scatter – Gather”)
 - ▶ **Up to 32 different addresses** with aligned access (to element) – “Parallel load”
 - ▶ **Up to 16 different addresses** with non - aligned access – “Bank Coupled” Parallel load

LSU and Memory sub-system

- ▶ CEVA-**XM** scatter-gather capability enables load/store vector elements from/into multiple memory locations in a single cycle
- ▶ Popular Use-Cases:
 - ▷ Look-Up Tables (LUT) - Duplicate a LUT for 32 different banks
 - ▷ Reduce system overhead by processing small data pieces from random locations:



XM Resources

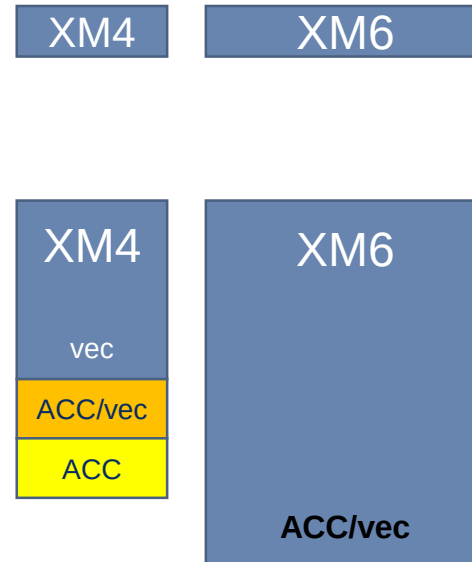
▶ Vector register file

▷ XM4

- ▶ 24 “pure” vectors
- ▶ 8 additional vectors that can be used also as 4 accumulators
- ▶ 8 “pure” accumulator vectors

▷ XM6

- ▶ 48 vectors or accumulator vectors
- ▶ Fully orthogonal
- ▶ Significant performance boost for “complex” kernels



XM6 – VPU Light

▶ 3rd VPU unit supports ALU operations

- ▷ Arithmetic
- ▷ Bit Manipulation
- ▷ Comparators
- ▷ Move, Permutation
- ▷ Type casting

▶ Not supported

- ▷ Multipliers
- ▷ Sliding window



VPU multiplication operations

Both VPU0 and VPU1 support vector multiplication operations, which include multiply and multiply-accumulate operations

Table 5-2. Vector Multiply Types

Vector Instructions	Source 1 Size	Source 2 Size	Number of Results	Result Size
vmxy, vmac	8 bits	8 bits	32	16 bits
	8 bits	16 bits	16	32 bits
	16 bits	16 bits	16	32 bits
	16 bits	32 bits	8	32 bits
	32 bits	32 bits	8	64 bit
vmad, vmac3	8 bits	8 bits	32	16 bits
	8 bits	16 bits	16	32 bits
	16 bits	16 bits	16	32 bits
	16 bits	32 bits	8	64 bit

VPU Computing Capabilities

- ▶ 128 8bit MACs
- ▶ 128 16bit MACs
- ▶ Up to 128 fixed-point 16x16 MAC Operations per cycle
 - ▷ The max utilization on MACs can be reached by sliding window instructions (ex: *vswgenmac*)
- ▶ Flexible operation types on vector
 - ▷ 8/16/32/64-bit native data types
 - ▷ Signed and unsigned full support

	Operations/Cycle
8x8 MAC	128
16x8 MAC	128
16x16 MAC	128
32x32 MAC	32
8b ALU	96
16b ALU	48
32b ALU	24
64b ALU	12

VPU Floating Point

- ▶ 32-bit Single-precision and 16-bit Half-precision support
- ▶ IEEE-754 standard fully compliant
 - ▷ Support the following rounding modes, according IEEE-754 standard:
 - ▶ Round Towards Even
 - ▶ Round Towards Zero
 - ▶ Round Towards Positive
 - ▶ Round Towards Negative
 - ▷ Verified .vs. Berkeley's reference C-model for single and half precision

FP Instruction set

Instruction	Description	Single Precision	Half Precision
		Vector Operations/Cycle*	
		XM6	XM6
VFPADD VFPSUB	Addition and subtraction of two FP numbers	8	16
VFPCMP VFPMIN VFPMAX	Compare, Min and Max of two FP numbers	8	16
VFPMPY	FP multiplication	8	16
VFPMAC	FP fused MAC	8	16
VFP-to-INT VFP-to-SHORT VFP-to-LONG	Type conversion to/from FP to Fixed point	8	16
VFPEXTRACT {MANT} VFPEXTRACT {EXPONENT} VFPEXTRACT {FPSIGN}	Extracts floating point to sign mantissa and Exponent	8	16
VFPCombine	Pack sign, mantissa and exponent to 32-bit floating point representation	8	16

* Same ISAs are supported in scalar (1 operation/cycle in each SPU)

Vector Non-Linear Instructions

- ▶ All functions are using table and then Linear approximation
- ▶ Optional configuration

Instruction	Description	Number of elements
		XM6
LOGE		1
EXPi	Exponent And Inverse	1
SQRT/ FP SQRT	Fixed point and FP Square Root	16
SQRTi/ FP SQRTi	Fixed point and FP Square Root And Inverse	1
ATAN		16
COS/SIN	Cosine and sine	1
TANH	Hyperbolic tangent	16
INV/ FP INV	Fixed point and FP Inverse (X) 1/X	16

XM4 vs. XM6 - Features Comparison

Feature	XM4	XM6
Vector size	256 bits	256 bits
Vector file size	40 partially orthogonal	48 fully orthogonal
Memory BW	2 Load / Store units 2 x 256 bits loads or 256 bits load + 256bit store	2 Load / Store units 2 x 256 bits loads 256 bits load + 256 bit store 512 bits Store (VHIST) in parallel
Memory partition	16 banks of 32 bits	16 banks of 32 bits
Scatter-Gather	32 addresses	32 addresses, enhanced capabilities such as auto stall on conflict
Scalar units	CEVA-X based	CEVA-X4 based
Vector units	2	3
Vector MAC fixed point	128 (16x8 or 8x8), 64 (16x16), 16 (32x32)	128 (16x8 or 8x8), 128 (16x16), 16 (32x32)
Scalar MAC fixed point	4 (16x16), 4 (32x32)	8 (16x16), 4 (32x32)
Vector Floating point math	16 MPY (32-bit)	16 MAC (32-bit) or 32 MAC (16-bit)
Scalar Floating point math	4 MPY (32-bit)	4 MAC (32-bit or 16-bit)
ALU units (8/16/32 bits)	64 / 32 / 16	96 / 48 / 24
Non-Linear Ops	Inv, Sqrt, Sqrt ⁻¹	Added: Sin, Cos, Arctan, TanH, In, Exp

▶ Vec-C Programming Language

Overview

- Achieving high-performance software is a complex task and sometimes the code can be very difficult to understand and maintain.
- Traditionally, optimized kernels were written in plain C with complex intrinsics or even assembly.
- The Vec-C language comes to help in the following levels:
 - Enables complex and maintainable code to be written in C++ (or even in a vector-C extension) without sacrificing performance.
 - Unleashes the power of the CEVA-XM family DSPs, enabling the code to be portable, readable, and maintainable for long lifecycles.

Overview

- Vec-C is based on data types, instructions, and operators, all of which are supported by CEVA-XM compilers as well as simulated on PC platforms using libraries.
- Writing Vec-C code is fairly simple because it looks like plain C++ code. Some architecture-specific instructions, data types, and operators have been added as language extensions, combined with an optimizing compiler to enable high performance.

Data Types - 256 Bits

CEVA Vec-C Types	GCC Vector Types	Description
256 Bits – Native Vectors		
char32	int8x32_t	32 elements of 8-bit signed two's complement integer
uchar32	uint8x32_t	32 elements of 8-bit unsigned integer
short16	int16x16_t	16 elements of 16-bit signed two's complement integer
ushort16	uint16x16_t	16 elements of 16-bit unsigned integer
int8	int32x8_t	8 elements of 32-bit signed two's complement integer
uint8	uint32x8_t	8 elements of 32-bit unsigned integer
long4	int64x4_t	4 elements of 64-bit signed two's complement integer
ulong4	uint64x4_t	4 elements of 64-bit unsigned integer
half16	float16x16_t	16 element of 16-bit floating-point
float8	float32x2_t	8 elements of float

Data Types - 128 Bits

CEVA Vec-C Types	GCC Vector Types	Description
128 Bits – Half Vectors		
char16	int8x16_t	16 elements of 8-bit signed two's complement integer
uchar16	uint8x16_t	16 elements of 8-bit unsigned integer
short8	int16x8_t	8 elements of 16-bit signed two's complement integer
ushort8	uint16x8_t	8 elements of 16-bit unsigned integer
uint4	uint64x4_t	4 elements of 32-bit unsigned integer
half8	float16x8_t	8 elements of 16-bit floating-point

Data Types - 512 Bits

CEVA Vec-C Types	GCC Vector Types	Description
512 Bits – Double Vectors		
short32	int16x32_t	32 elements of 16-bit signed two's complement integer
ushort32	uint16x32_t	32 elements of 16-bit unsigned integer
int16	uint32x16_t	16 elements of 32-bit signed two's complement integer
uint16	int32x16_t	16 elements of 32-bit unsigned integer
long8	int64x8_t	8 elements of 64-bit signed two's complement integer
ulong8	uint64x8_t	8 elements of 64-bit unsigned integer

Packing and Unpacking

The CEVA-XM6 core is designed for optimal performance with 256-bit vectors. In many applications, longer or shorter vectors must be used to either increase accuracy (because of higher moment calculations in fixed-point) or reduce memory usage. To do this, the compiler has built-in operators for packing and unpacking vectors.

The following code example demonstrates the vpack/vunpack operation prototypes (applicable to short8, short16, short32, int8, and int16 as well):

- `char32 vpack(char16 inA, char16 inB);`
- `char16 vunpack_lo(char32 inA);`
- `char16 vunpack_hi(char32 inA);`

The following code example demonstrates how multiple vectors are concatenated to create one vector:

- `z = vpack(a,b);`

char16	a	97	41	78	19	20	97	8	20	50	75	50	42	49	11	76	74																
char16	b	61	82	27	53	70	97	19	60	45	34	74	65	37	29	45	15																
char32	z	61	82	27	53	70	97	19	60	45	34	74	65	37	29	45	15	97	41	78	19	20	97	8	20	50	75	50	42	49	11	76	74

Unpacking does the reverse operation, as demonstrated in the following code example:.

- `a = vunpack_lo(z);`
- `b = vunpack_hi(z);`

Casting

The CEVA-XM compiler does not perform implicit-type conversions on vector data types. The reason for the lack of automatic casting is to protect the programmer and ensure that the Vec-C instruction that is used correlates to the DSP instruction set.

In Vec-C code, instructions must be called with the correct types; otherwise, the compiler assumes an error and requires the programmer to explicitly make the conversion

The following code example demonstrates a vector declaration:

- `short8 my_s8 = 0;`
- `int8 my_i8 = my_s8; // not allowed`
- `int8 yours_i8 = (int8)my_s8; // allowed`

The CEVA-XM6 instruction set supports signed (sat) and unsigned (usat) saturation using the Vcast instruction, as demonstrated in the following code example:

- `short8 yours_s8 = vcast(sat,my_i8)`

Instruction Syntax

Vec-C instruction prototypes are in the form of a C-style function call, as demonstrated in the following code example:

```
int16 vmac3(subtract_sw in_subtract,  
            char16 inA,  
            unsigned short inB,  
            short16 inC,  
            unsigned short inD,  
            int16 inoutW);
```

Where:

- **in_subtract** is the instruction's optional switch.
- **inA ... inF** are used for the input vectors.
- **inoutW** is used as an input-output vector (equivalent to the C++ by-reference).

The compiler selects the most appropriate assembly instruction based on the data type of the arguments used in the code.

Operators

The simplest (and sometimes most efficient) way to use the vector data types is with overloaded operators that are built into the Vec-C compiler.

The following table lists the Vec-C overloaded operators. These operators can be used with all vector types, enabling the compiler to choose the most appropriate CEVA-XM opcode to execute the operation.

Operator	Description
+, +=	Vector elements SIMD addition (<i>vadd</i>)
-, -=	Vector elements SIMD subtraction (<i>vsub</i>)
*, *=	Vector elements SIMD multiplication (<i>vmpy</i> , <i>vmac</i>)
>>, >>= <<,<<=	Vector elements SIMD shift left or right (<i>vshiftrl</i> , <i>vshiftr</i>)
^ &	Vector elements SIMD logical operations (<i>vor</i> , <i>vand</i> , <i>vxor</i> , <i>vnot</i>)
== != > < >= <=	Vector comparison operators

The following code example demonstrates how to use operators. It is a snippet of the main loop of the RGB to YUV kernel, implemented using operators only:

```
uchar32 *pR, *pG, *pB;
uchar32 *pU, *pV, *pY;
for (i = 0; i < len; i += 32) {
    uchar32 vR = *pR++;
    uchar32 vG = *pG++;
    uchar32 vB = *pB++;
    uchar32 vY = (uchar32)(((short32)vB * 25 + (short32)vG * 129 + (short32)vR * 66 + (128 + 16 * 256)) >> 8);
    *pU++ = (uchar32)(((short32)vY * (-147) + (short32)vB * 126 + 128 + 128 * 256) >> 8);
    *pV++ = (uchar32)(((short32)vY * (-186) + (short32)vR * 160 + 128 + 128 * 256) >> 8);
    *pY++ = vY;
}
```

Vector Predicated Execution

Vector Predictor Registers (VPRs) are special registers in the CEVA-XM core and are accessible via both scalar and vector units.

They are 32-bit/16-bit/8-bit register sets that determine which of the vector elements will be affected by an instruction execution (that is, written back to the output vector), and are referenced as **[vprX.b32]** in the architecture specification.

The equivalent of this operator in C is the ternary-operator:

<condition> ? <if_true> : <if_false>

Using a VPR is essential when implementing algorithms that require accurate selection of elements, for example:

- Selective filtering
- Data compactness
- Merging and sorting
- Splitting vector processing into groups (positive/negative, odd/even, below/above thresholds, and so on)

Vector Predicated Execution

The syntax for using a VPR in Vec-C is the **vselect()** statement. The instruction prototype for selecting an eight-integer vector is:

```
int8 vselect(int8 inA, int8 inB, unsigned char inC);
```

The resulting vector will contain elements from either inA or inB, depending on the value of the corresponding bit in the VPR (inC). When the bit is set, the element is taken from inB; otherwise, it is taken from inA.

The following code example demonstrates the vselect() statement:

```
int8 c, b, odd, msb;  
c = 0xC; // {C, C, C, C, C, C, C, C}  
b = 0xB; // {B, B, B, B, B, B, B, B}  
odd = vselect(c, b, 0xAA); // 0b10101010  
// {B, C, B, C, B, C, B, C}  
msb = vselect(c, b, 0xF0); // 0b11110000  
// {B, B, B, B, C, C, C, C}
```

▶ LSU Vector Instruction Set

Load/Store Unit

Instructions

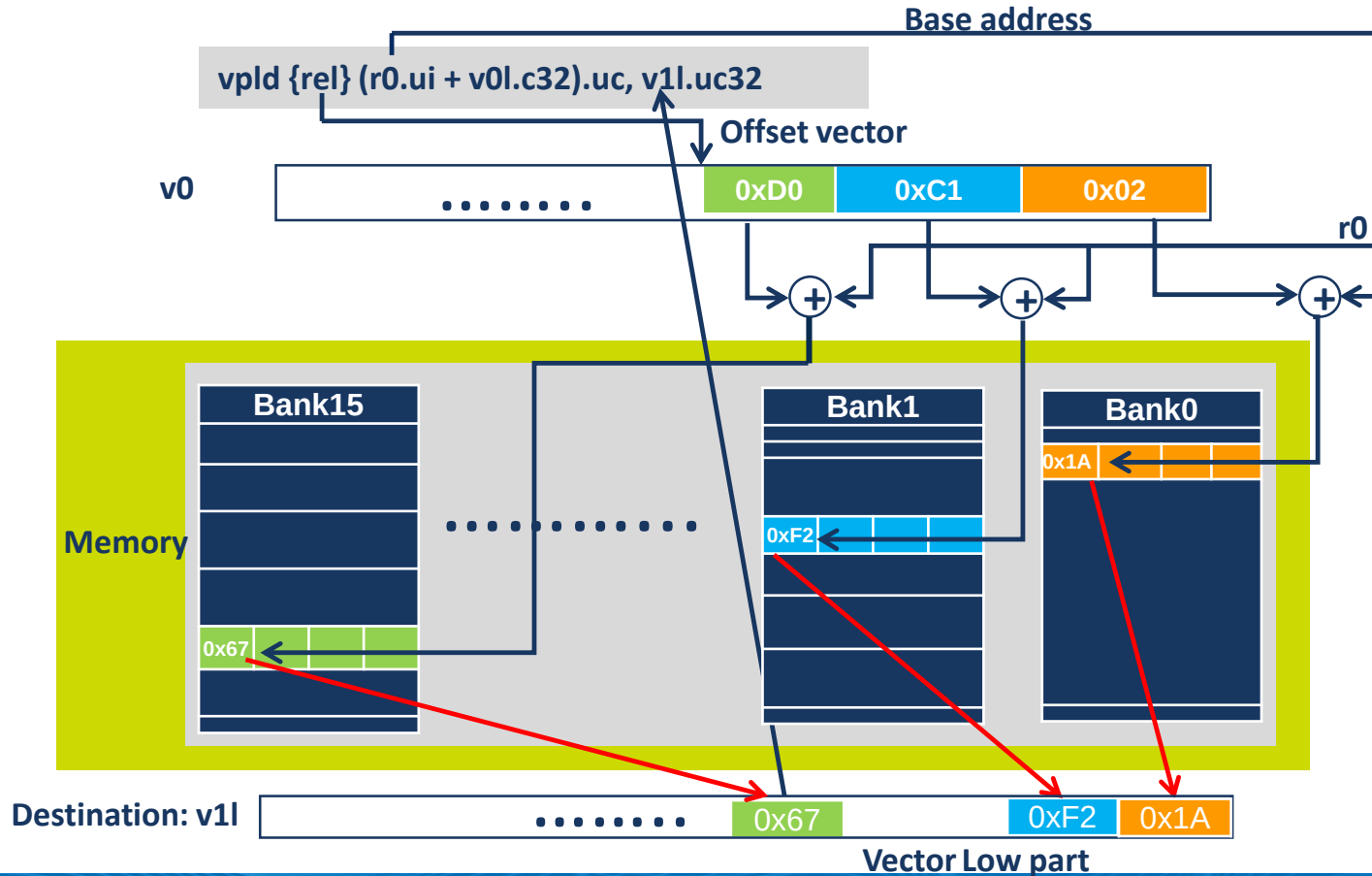
- ▶ LSU - Instruction Set :
 - ▷ *ld* instruction: loads memory operands from the memory into core register(s)
 - ▷ *st* instruction: stores core register(s) into the data memory
 - ▷ *pop* instruction (LS0 only): pops operand from the software stack into core register(s)
 - ▷ *push* instruction (LS1 only): pushes core register(s) onto the software stack
 - ▷ *modr* instruction: modifies a pointer

Load/Store Unit

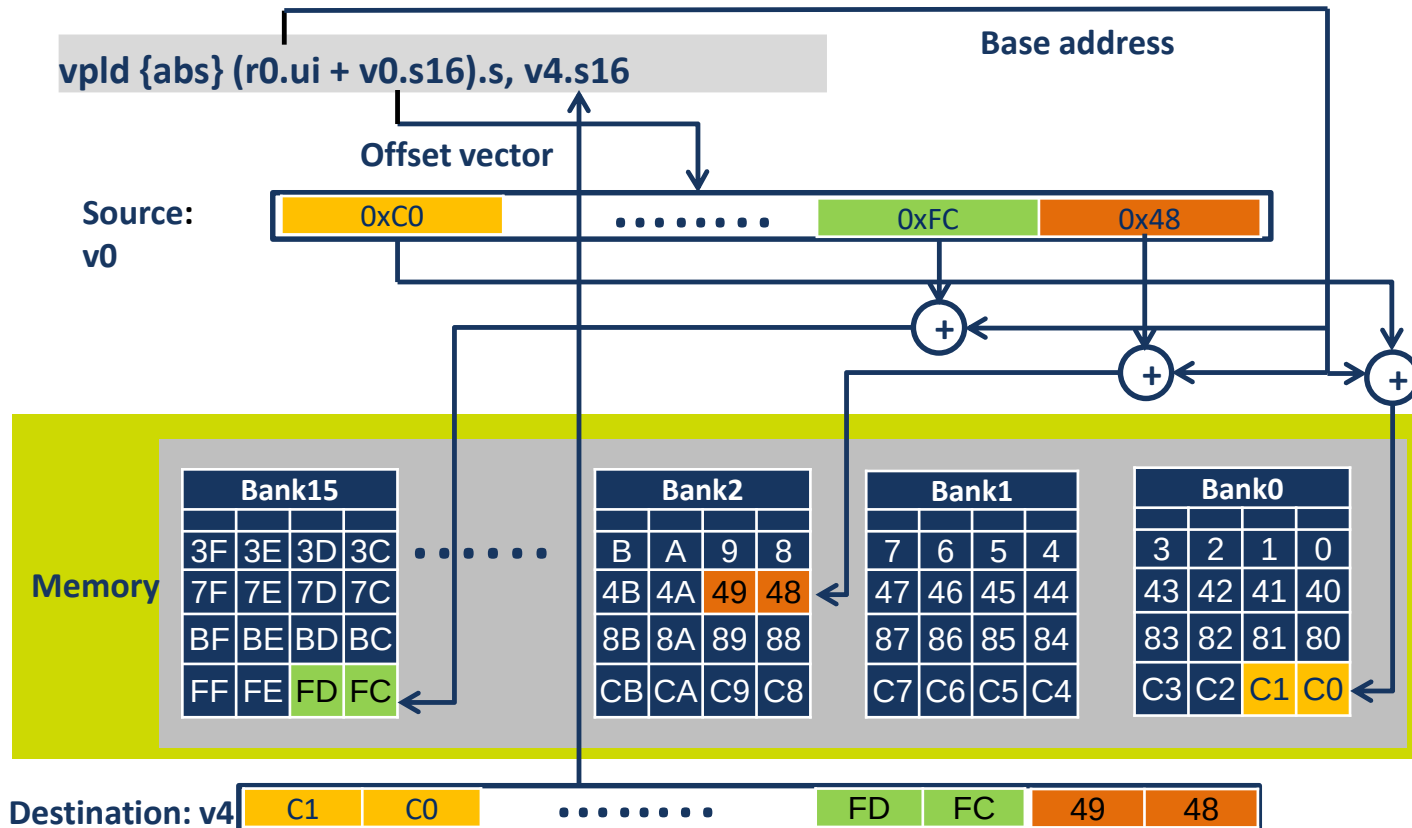
Instructions (Cont.)

- ▶ LSU Vector Instruction Set:
 - ▶ *vld/vst*: loads/stores 8 double-words/16 words/32 bytes from the memory into 256 bit vector register(s)
 - ▶ *vpld/vpst*: loads/stores up to 16 double word(s) from the memory into vector register(s) using 16 addresses using vector of addresses
 - ▶ *vbcpld/vbcpst*: load/stores eight elements of non-aligned double words/ words / bytes
 - ▶ *vpop*: Pop operands from the software stack into the VPU vector registers
 - ▶ *vpush*: Push VPU vector registers into the software stack

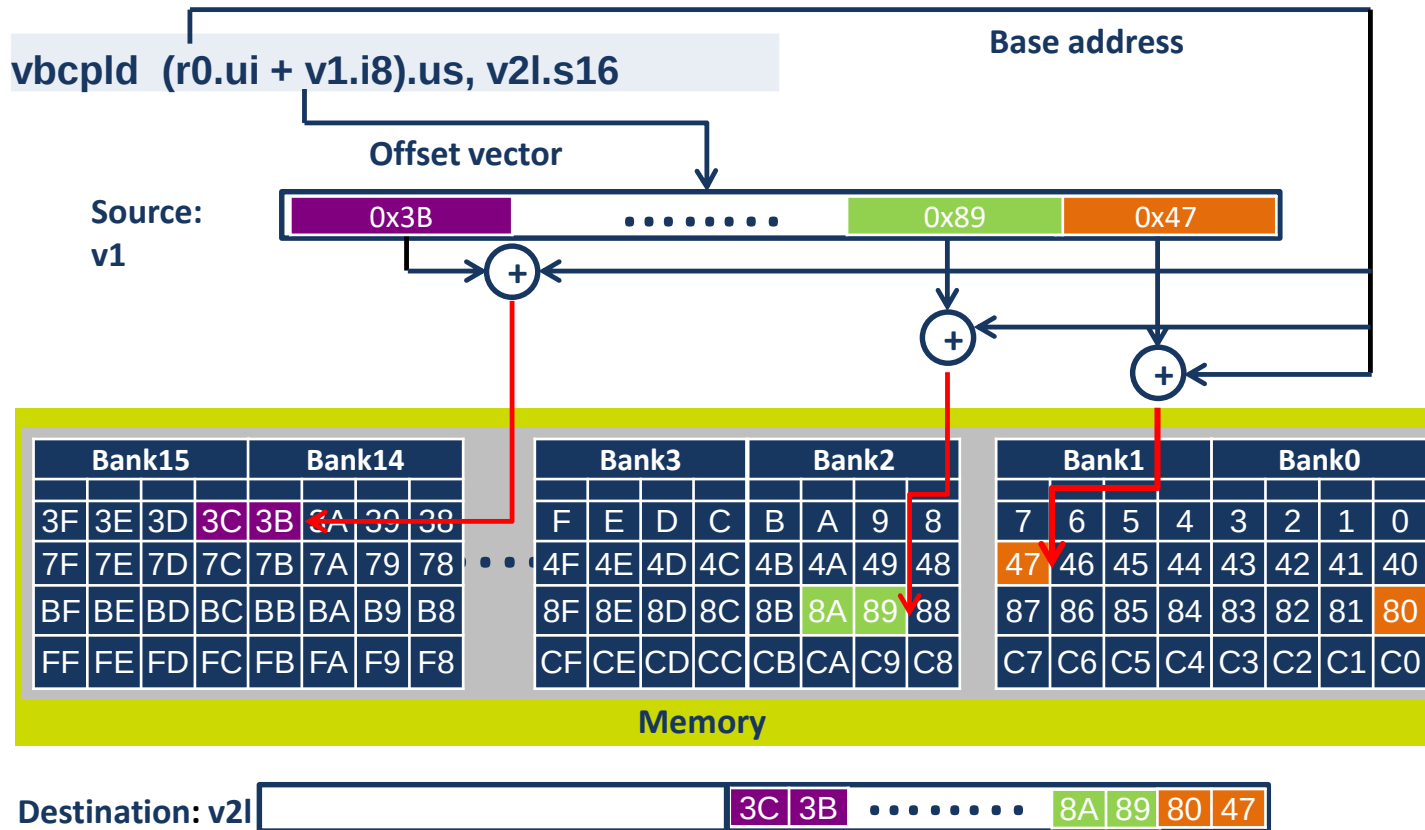
Vector parallel Load



Vector parallel Load- Absolute



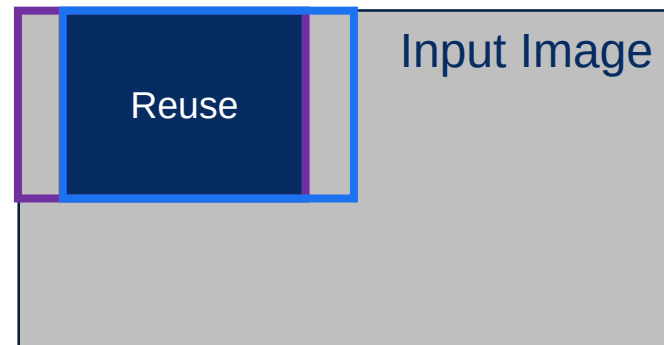
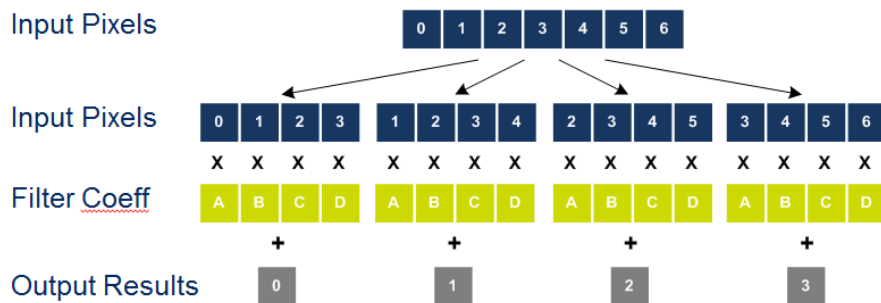
Parallel Id Coupled Banks (vbcpld)



▶ Sliding window

Overview

- ▶ The Sliding-Window Mechanism takes advantage of pixel overlap in image processing by reusing same data to produce multiple outputs.
 - ▷ Significantly increases processing capability
 - ▷ Reduces power consumption



Vector Sliding Window Instructions

▶ Sliding-window sources include:

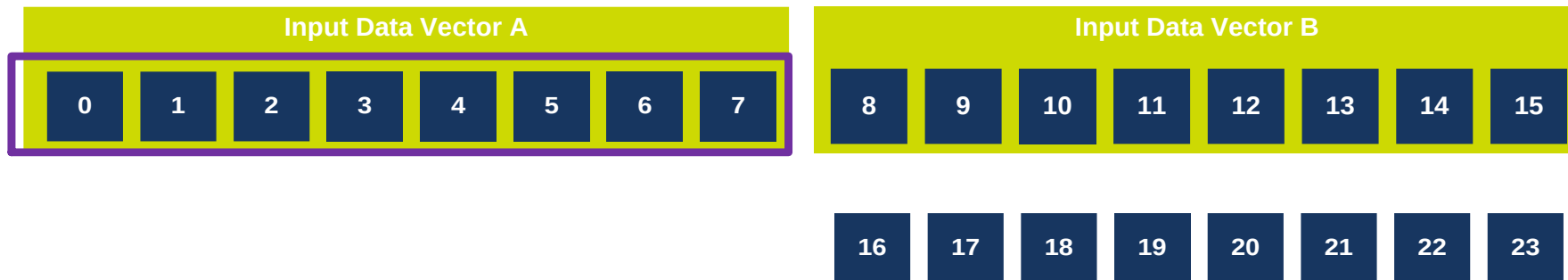
1. Two input data vectors containing the elements to be processed
2. A third and fourth input vector containing filter coefficients.

▶ A sliding window operation is performed in three stages:

1. Up to four data elements are selected from the two input data input vectors. Up to four elements are selected from the third coefficient vector.
2. An arithmetic operation is performed on the data elements.
This may be multiplication, absolute difference or subtraction between the selected data elements and coefficient elements.
3. The resulting products/differences are added together (up to four results) and accumulated with the destination vector.

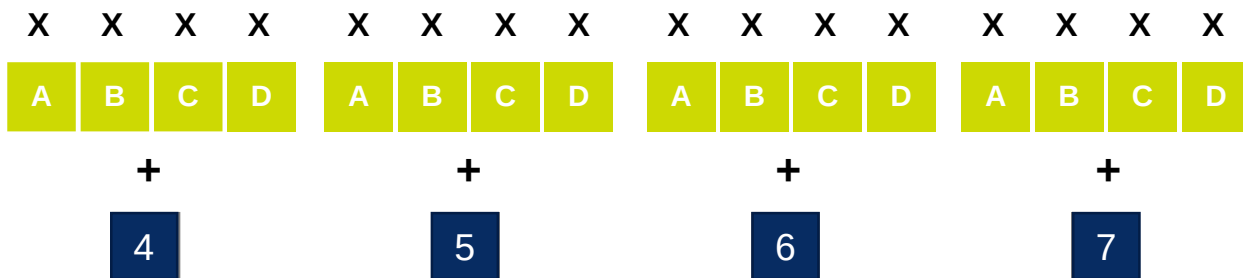
Sliding Window (SW) Mechanism

- ▶ SW resources include
 - ▶ Two input data vectors containing the elements to be processed
 - ▶ A third input vector containing filter coefficients
 - ▶ Output Accumulator / Vector containing the results
- ▶ The SW operation considers the two input data vectors as one concatenated source, thus enabling the instruction to *slide* from one vector to the next seamlessly



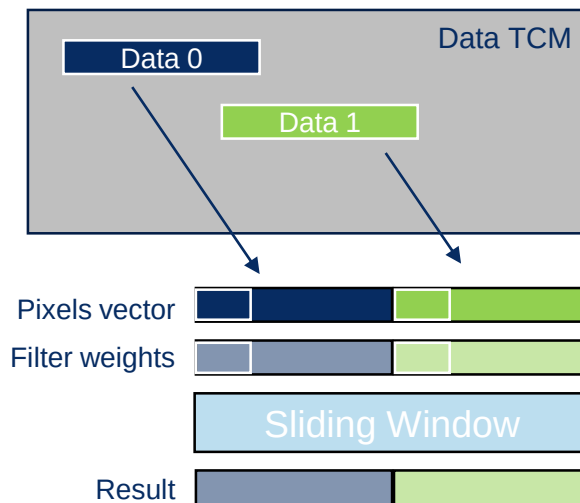
Input Data Reuse Example

- ▶ The vector instruction performs the arithmetic operation multiple times in parallel (producing eight, 16 or 32 results). The next sliding-window operation is offset by a step from the previous operation.



Sliding-Window Slice Mechanism

- ▶ CEVA-XM6 introduce a new concept of **slices** – allowing a SIMD instruction to perform difference operations for different parts of the vector
- ▶ SIMD operation can be split to 2/4/8 slices
- ▶ Slice feature allows processing small input using a wide vector register file (256-bit)



VSWGEMAC Configuration (1)

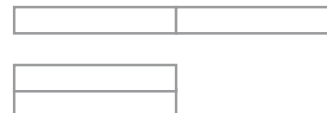
Standard fields

- ▶ Starting offset data – starting point on processed data
- ▶ Starting offset coeff – starting point of used coefficients
- ▶ Pattern – eight bits mask that describe which data elements are used in a 2x4 window.
- ▶ Step size – describes how many elements to skip until the next pattern
- ▶ window of the next calculation
- ▶ Post shift/init value – post shift when writing to vector (and not accumulator) and init value when init switch is used
- ▶ Number of filters in a slice – divide the slice output to Y parts, each part will be processed using a different filter (Y is power of 2)

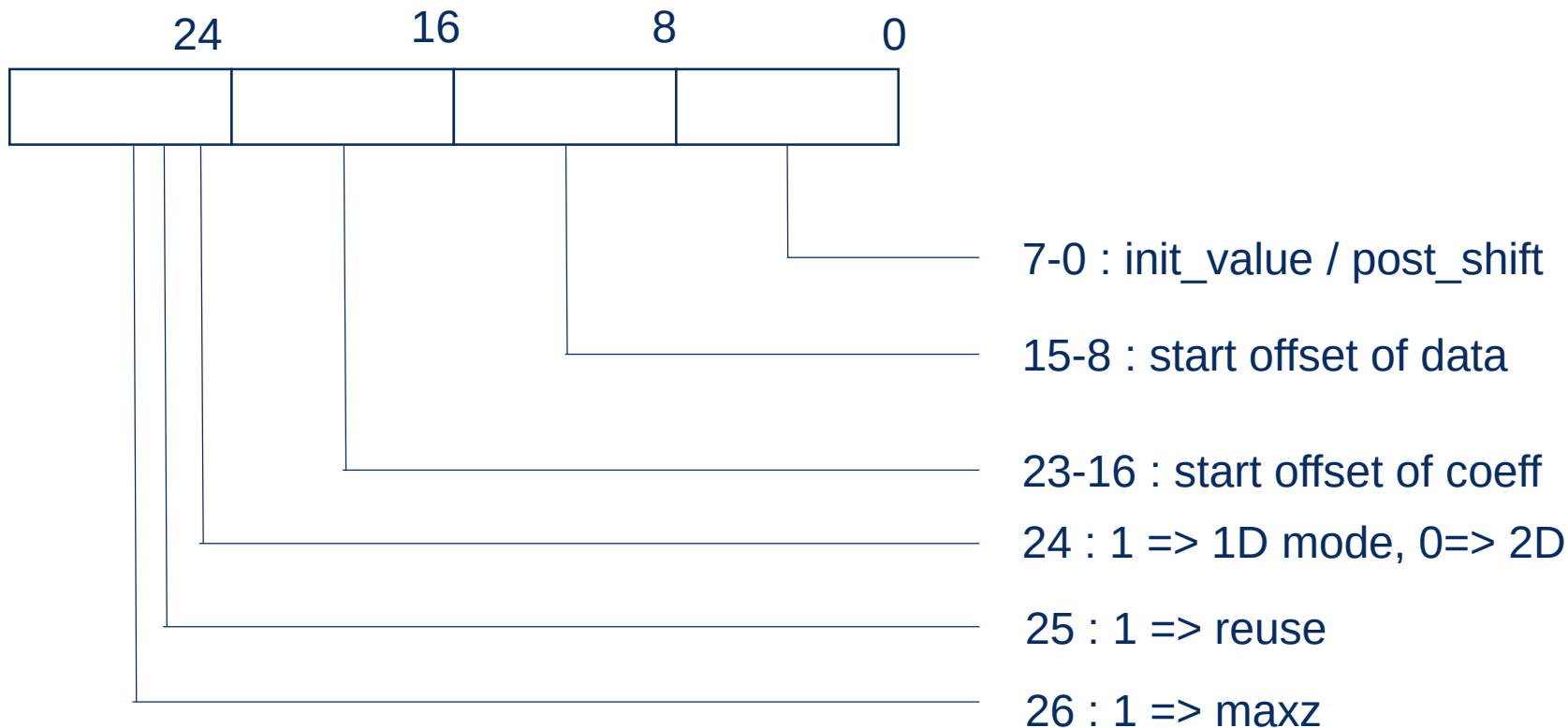
VSWGEMAC Configuration (2)

New fields

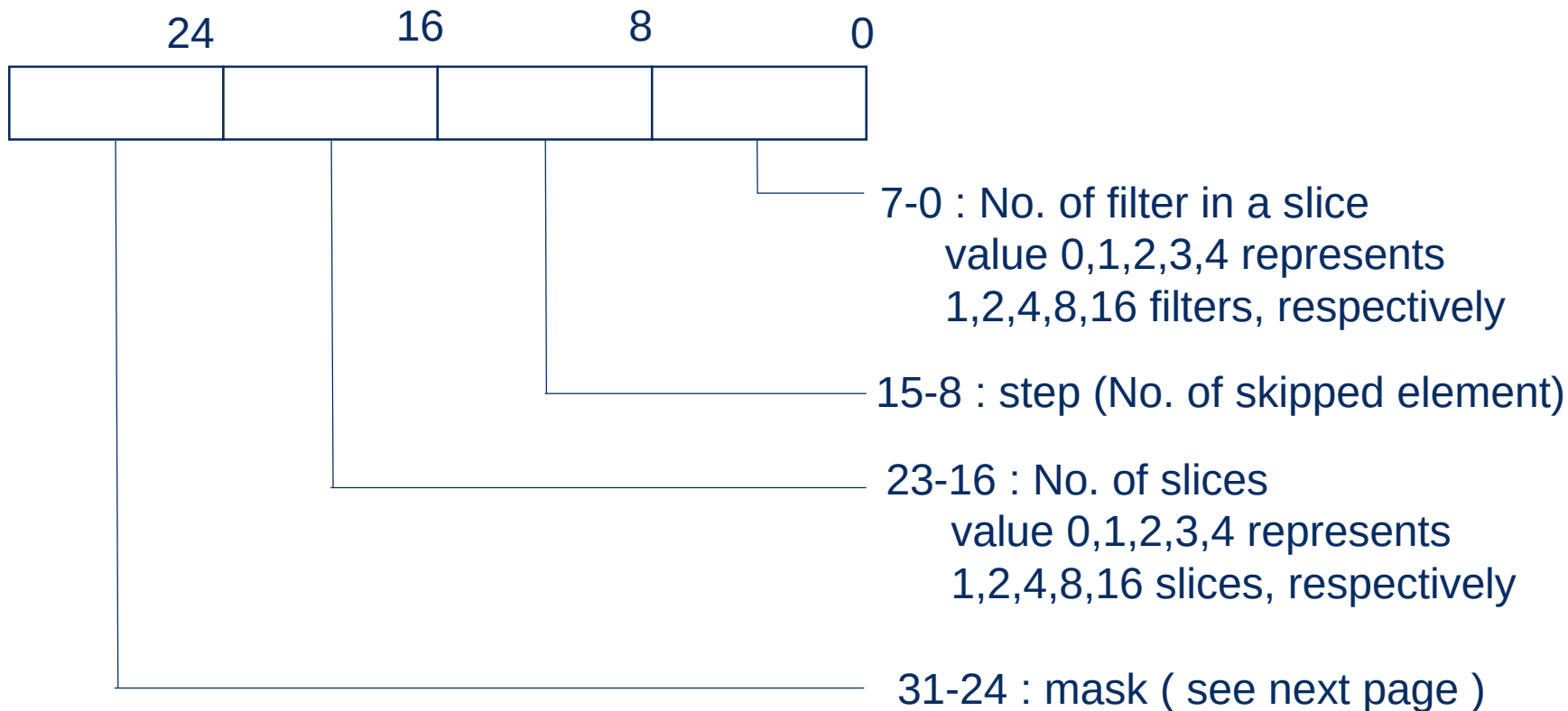
- ▶ Number of Slices – Divide the input and output to X parts. Each part is processed independently (X is power of 2)
- ▶ Mode:
 - ▷ Dimension
 - ▶ 1D – 2 input vectors are treated as one alongside the other
 - ▶ 2D – 2 input vectors are treated as one above the other
 - ▷ Reuse coefficients:
 - ▶ Yes – use the same filter coefficients, for each slice
 - ▶ No – each slice uses a different set of filter coefficients
 - ▷ Maxz (max with zero):
 - ▶ Yes – Substitute all negative values in the result register, with zero.
 - ▶ No – do not change values before writing to vector
- ▶ Coefficient are taken sequentially from the coefficient vectors



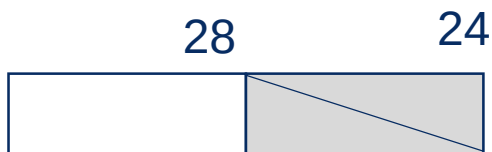
Configuration : parameter register 1



Configuration : parameter register 2



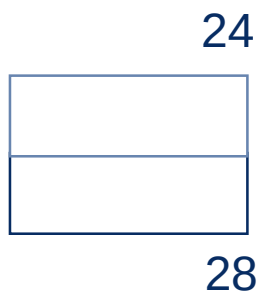
parameter register 2[31:24] (mode = 1D)



► In 1D mode

- ▷ bit [27:24] are ignored
- ▷ bit [31:28] represents the bit mask for the 4 taps filter.
 - bit 28: first tap
 - bit 29 : second tap
 - bit 30 : 3rd tap
 - bit 31 : fourth tap

parameter register 2[31:24] (mode = 2D)



▶ In 2D mode

▷ bit [31:28] represents the bit mask for the row 1 (vecA).

▶ bit 28: first tap

▶ bit 29 : second tap

▶ bit 30 : 3rd tap

▶ bit 31 : fourth tap

▷ bit [27:24] represents the bit mask for the row 2 (vecB)

▶ bit[31:24] figures 4x2 2D pattern.

▷ 4 elements out of 8 elements can be chosen by bit mask.

Configuration Via Macros

▶ Define Macros

▷ Setting Mode Registers

```
#define SWGEN_SETMODE(dimension,reuse,clip_neg)    (dimension | reuse << 1 | clip_neg << 2)
```

▷ Creating Default Mode Configuration

```
#define SWGEN_BASE_MODE_1D                        SWGEN_SETMODE(SWGEN_1D,SWGEN_NO_REUSE,SWGEN_NO_CLIP_NEG)
```

▷ Define Configuration Control

```
#define SWGEN_CTRL0(init_psh,src_offset,coeff_offset,sw_mode)
```

```
((init_psh) & 0xff) << SWGEN_INIT_PSH_VAL | ((src_offset) & 0xff) << SWGEN_SRC_OFFSET | ((coeff_offset) & 0xff) << SWGEN_COEFF_OFFSET | ((sw_mode) & 0xff) << SWGEN_MODE )
```

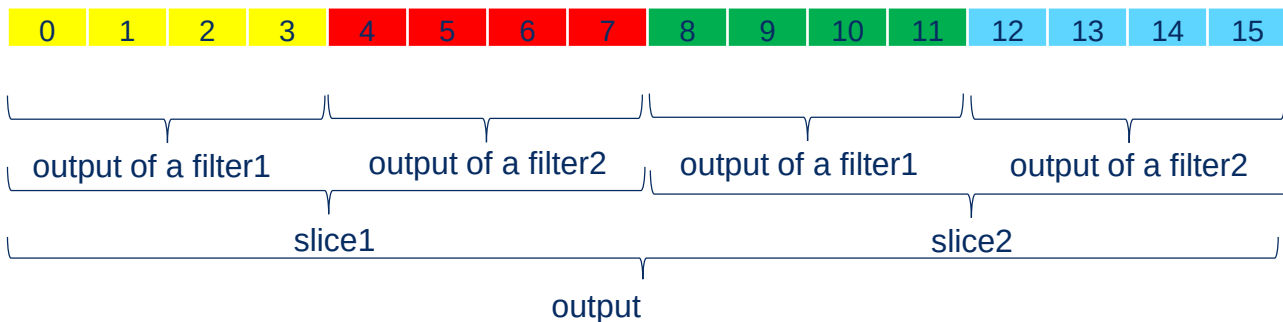
▶ Use Macro

```
uint ctrl0 = SWGEN_CTRL0(0, 0, 0, SWGEN_BASE_MODE_1D);
```

```
vacc0 = vswgenmac(init, v0, v0, v_coeff, v_coeff, ctrl0, cline);
```

Output division according to configuration (1)

- ▶ Number of results
 - ▷ No. results per slice = output size / No. of slices
 - ▷ In each slice the No. of results per filter = No. of results per slice / No. of filters
- ▶ 128 16x16 MACs in VPU0/1 can generate 32 results per cycle
 - ▷ $2 \text{ VPUs} * 16 \text{ results/VPU} * 4 \text{ MAC/result} = 128 \text{ MACs}$



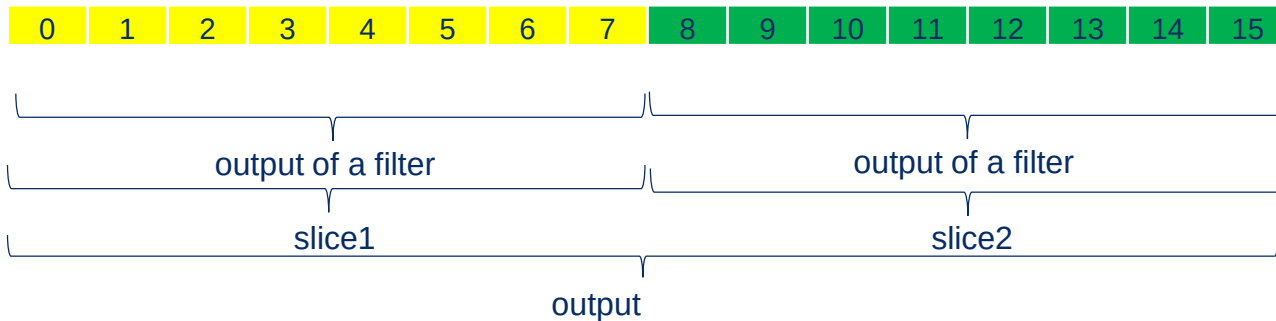
No of filter in a slice is 2, in this example

No of slice is 2, in this example

Output division according to configuration (2)

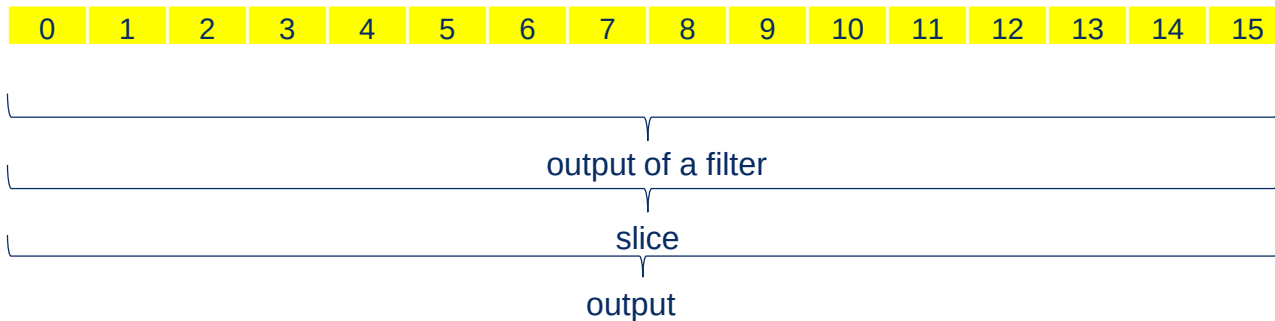
No of filter in a slice is 1, in this example

No of slice is 2, in this example



No of filter in a slice is 1, in this example

No of slice is 1, in this example

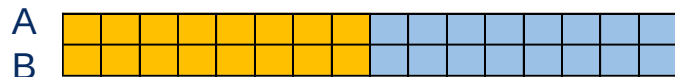


2D/1D Modes and slices

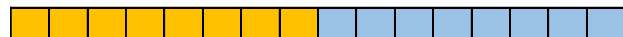
Divide the input to separate processed units

▶ **2D and 2 slices:**

▶ Input vectors



▶ Output vector



▶ **1D and 4 slices:**

▶ Input vectors



▶ Output vector



Multi-scalar bilinear interpolation

16bitx16bit -> 16 32bit results

Data offset = 0

Slices=2

Step=0

Number of filters=1

Mode = 2D No reuse

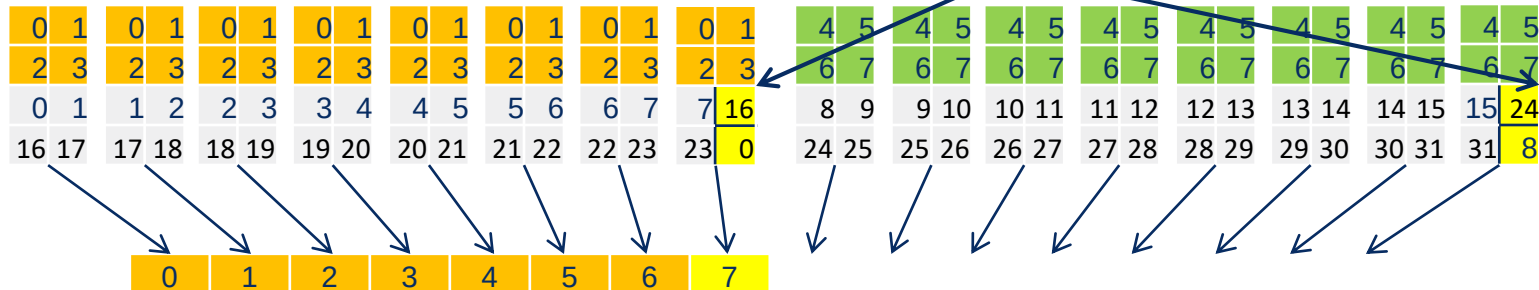
vC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

vA 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
vB 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Pattern

x	x		
x	x		

Wrap around per slice



Multi-scalar bilinear interpolation

16bitx16bit -> 16 32bit results

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

Pattern

x	x		
x	x		

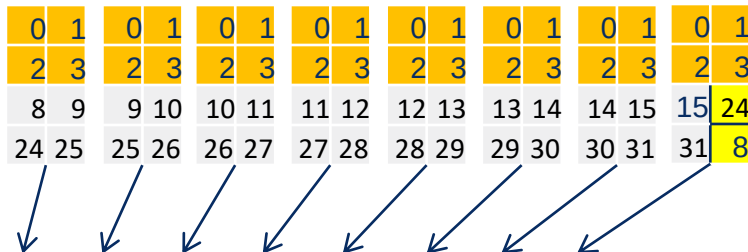
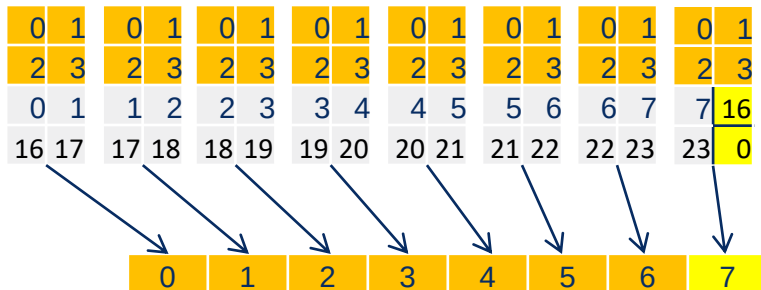
Data offset = 0

Slices=2

Step=0

Number of filters=1

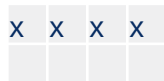
Mode = 2D No reuse



Multi-filter

16bitx16bit -> 16 32bit results

Pattern



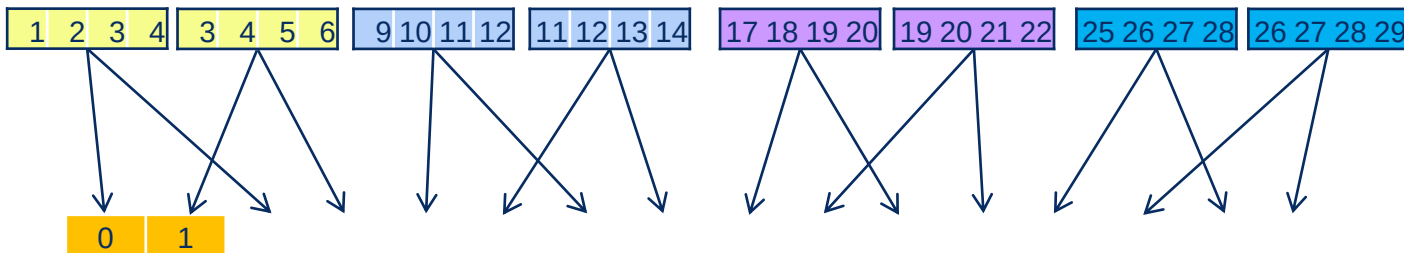
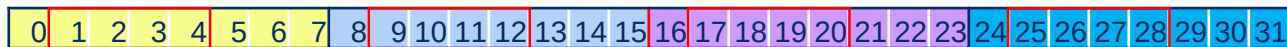
Slices=4

Number of filters=2

Step=1

Mode = 1D Reuse coeff

Data offset=1



Other Sliding-Window Instructions

► VSWGENSAD

$$z[k] = z_{i-1}[k] + \sum_{n=0}^3 |t[q+n] - X[r+(1+s)k+P_n]|$$

► VSWGENUBCMP – used in SigmaFilter

$$z[k] = z_{i-1}[k] + \sum_{n=0}^3 \begin{cases} src & |c - src| < th \\ 0 & |c - src| \geq th \end{cases}$$

► VSWGENSUB – used in bilateral filter

$$z[k] = \sum_{n=0}^3 (x[r+k+n] - t[k]) \begin{cases} th \\ -th \end{cases}$$

argument	description
$z_i[]$	Result accumulator (inoutW) of iteration i
$X[r]$	Input slice [inA, inB], starting at position r. in 2D mode X is matrix
$t[q]$	Target vector [inC, inD], starting at position q
P_n	2D position defined by the pattern
s	Number of elements to skip
k	Index of the element position in the output vector

argument	description
$z[]$	Result vector
$x[r]$	Input slice [inA, inB], starting at position r
$t[]$	Coefficient vector [inC], starting at position q
sat	Number of bits for saturation value calculation
th	$2^{sat} - 1$
k	Index of the element position in the output vector

Thank You



www.ceva-dsp.com

Copyright © 2020 CEVA, Inc. and/or its subsidiaries. All rights reserved.
All specifications are subject to change without notice.